TEMA 3. Excepciones

1. Empecemos un tema sobre control de errores en lenguajes de programación, con algo básico. En C, donde no existen las excepciones, pongamos un ejemplo de una raíz que toma número flotante positivo. Queremos controlar el error si la función recibe un número negativo. El usuario debe ser informado pero desde fuera de la función `raiz` ¿Cómo indicamos ese error?. Enumera dos opciones diferentes de diseñar, poniendo un ejemplo de código de cada una.

Respuesta

En C, donde no existen excepciones, el control de errores se puede realizar mediante el uso de valores de retorno especiales o modificando una variable externa. Ambas estrategias permiten informar al usuario sobre el error desde fuera de la función.

1. **Usar un valor de retorno especial:**
   En este enfoque, la función devuelve un valor específico (por ejemplo, `-1`) para indicar que ocurrió un error. El código llamador debe verificar este valor para manejar el error.

```c
#include <stdio.h>
#include <math.h>

double raiz(double numero) {
    if (numero < 0) {
        return -1; // Indica error
    }
    return sqrt(numero);
}

int main() {
    double resultado = raiz(-4);
    if (resultado == -1) {
        printf("Error: número negativo\n");
    } else {
        printf("Resultado: %f\n", resultado);
    }
    return 0;
}
```

2. **Usar una variable externa para el estado del error:**
   Aquí, se utiliza una variable global o pasada por referencia para indicar si ocurrió un error. Esto permite que la función devuelva el resultado calculado normalmente.

```c
#include <stdio.h>
#include <math.h>

int error = 0; // Variable global para el estado del error

double raiz(double numero) {
    if (numero < 0) {
        error = 1; // Indica error
        return 0;
    }
    error = 0;
    return sqrt(numero);
}

int main() {
    double resultado = raiz(-4);
    if (error) {
        printf("Error: número negativo\n");
    } else {
        printf("Resultado: %f\n", resultado);
    }
    return 0;
}
```

Ambos métodos tienen limitaciones. Por ejemplo, el uso de valores de retorno especiales puede entrar en conflicto con valores válidos, y el uso de variables externas puede dificultar el manejo de errores en programas concurrentes.

2. Brevemente ¿Qué es una "excepción"? ¿Con qué objetivo las usa un programador cuando implementa funciones o cuando las llama?

Respuesta

Una excepción es un mecanismo que permite manejar errores o situaciones inesperadas que ocurren durante la ejecución de un programa. En lugar de depender de valores de retorno o variables

externas, las excepciones interrumpen el flujo normal del programa y transfieren el control a un bloque de código diseñado para manejar el error.

El objetivo principal de las excepciones es separar la lógica principal del programa del manejo de errores, lo que mejora la claridad y el mantenimiento del código. Cuando un programador implementa funciones, puede usar excepciones para señalar errores que no pueden resolverse dentro de la función. Por otro lado, al llamar a funciones, el programador puede capturar y manejar estas excepciones para garantizar que el programa continúe funcionando de manera controlada.

Por ejemplo, en Java, una excepción puede lanzarse cuando se intenta dividir por cero o acceder a un índice fuera de los límites de un arreglo. Esto permite que el programador maneje estos casos sin mezclar la lógica de manejo de errores con la lógica principal del programa.

3. Reescribe el mismo ejemplo de raíz, pero en Java, metiendo ese método en una clase `Calculadora` y llama a dicho método desde el método `main`, mostrando cómo se puede controlar desde fuera.

Respuesta

En Java, el manejo de errores se realiza mediante excepciones. A continuación, se muestra cómo implementar el ejemplo de la raíz cuadrada en una clase `Calculadora` y manejar el error desde el método `main`.

```java
class Calculadora {
    public double raiz(double numero) throws IllegalArgumentException {
        if (numero < 0) {
            throw new IllegalArgumentException("El número no puede ser negativo");
        }
        return Math.sqrt(numero);
    }
}

public class Main {
    public static void main(String[] args) {
        Calculadora calc = new Calculadora();
        try {
            double resultado = calc.raiz(-4);
            System.out.println("Resultado: " + resultado);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

En este ejemplo, el método `raiz` lanza una excepción `IllegalArgumentException` si el número es negativo. El método `main` utiliza un bloque `try-catch` para capturar y manejar esta excepción, mostrando un mensaje de error al usuario. Este enfoque permite separar la lógica de cálculo del manejo de errores, mejorando la claridad del código.

4. ¿Qué es "lanzar" una excepción? ¿Qué es "controlar" o "capturar" una excepción? ¿Qué es que se "propague" una excepción? ¿Qué le va ocurriendo a las funciones en la pila de llamadas por donde se va propagando la excepción? ¿Las funciones que no la controlan se reanudan después

de alguna forma? Explica con el mismo ejemplo anterior en Java de la raíz cuadrada.

Respuesta

Lanzar una excepción significa interrumpir el flujo normal del programa y crear un objeto de excepción que contiene información sobre el error. Este objeto se transfiere a un manejador de excepciones, que puede estar en el mismo método o en uno superior en la pila de llamadas.

Controlar o capturar una excepción implica usar un bloque `try-catch` para interceptar la excepción y manejarla de manera adecuada. Si no se captura, la excepción se propaga hacia arriba en la pila de llamadas, buscando un manejador en los métodos que la invocaron.

Cuando una excepción se propaga, las funciones en la pila de llamadas se terminan abruptamente y no se reanudan. En el ejemplo de la raíz cuadrada, si el método `main` no capturara la excepción, esta se propagaría hasta el sistema, terminando el programa con un mensaje de error.

```java
public class Main {
    public static void main(String[] args) {
        Calculadora calc = new Calculadora();
        try {
            double resultado = calc.raiz(-4);
            System.out.println("Resultado: " + resultado);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

En este caso, el bloque `catch` captura la excepción y evita que se propague más allá del método `main`. Esto permite manejar el error de manera controlada y evitar que el programa termine abruptamente.

5. ¿Qué ventajas tiene frente a C, la "propagación natural" de las excepciones a través de la pila (*stack*) de llamadas?

Respuesta

La propagación natural de excepciones en lenguajes como Java ofrece varias ventajas frente a C, donde no existe este mecanismo. En C, el manejo de errores suele depender de valores de retorno o variables globales, lo que puede complicar el código y hacerlo menos legible.

En Java, las excepciones permiten que los errores se propaguen automáticamente a través de la pila de llamadas, sin necesidad de verificar manualmente los valores de retorno en cada nivel. Esto simplifica el código, ya que los programadores pueden centrarse en la lógica principal y manejar los errores solo donde sea necesario.

Además, las excepciones en Java incluyen información detallada sobre el error, como el tipo de excepción y la traza de la pila, lo que facilita la depuración. En C, el programador debe implementar manualmente mecanismos para registrar y transmitir esta información, lo que puede ser propenso a errores.

Por último, la propagación de excepciones permite un manejo más estructurado y modular de los errores, ya que los bloques `try-catch` pueden colocarse en puntos estratégicos del programa, en lugar de dispersar verificaciones de errores en todas partes.

6. En orientación a objetos, ¿las excepciones suelen ser objetos? ¿Qué ventajas tiene esto en términos de encapsulación? ¿Podemos entonces crear excepciones personalizadas?

Respuesta

En los lenguajes orientados a objetos, como Java, las excepciones suelen ser objetos que heredan de la clase base `Throwable`. Esto permite que las excepciones aprovechen las características de la orientación a objetos, como la herencia y el polimorfismo.

El uso de excepciones como objetos ofrece ventajas significativas en términos de encapsulación. Por ejemplo, una excepción puede contener información detallada sobre el error, como un mensaje descriptivo, un código de error y la traza de la pila. Esto facilita la depuración y el manejo de errores de manera estructurada.

Además, es posible crear excepciones personalizadas definiendo clases que hereden de `Exception` o `RuntimeException`. Esto permite representar errores específicos de la lógica del programa, mejorando la claridad y la mantenibilidad del código. Por ejemplo:

```
class MiExcepcion extends Exception {
    public MiExcepcion(String mensaje) {
        super(mensaje);
    }
}
```

7. En relación con las ventajas de la encapsulación, comparando el ejemplo en C con Java. ¿Qué **información esencial** lleva cualquier **objeto excepción** que es muy útil tener cuando se llega a un manejador?

Respuesta

En Java, los objetos de excepción encapsulan información esencial que facilita el manejo de errores. Esto incluye:

1. **Mensaje descriptivo:** Proporciona detalles sobre la naturaleza del error, lo que ayuda a identificar rápidamente el problema.
2. **Traza de la pila:** Muestra la secuencia de llamadas que llevaron al error, facilitando la depuración.
3. **Tipo de excepción:** Permite distinguir entre diferentes tipos de errores y manejar cada uno de manera específica.

En comparación, en C, el manejo de errores suele depender de valores de retorno o variables globales, lo que limita la cantidad de información que se puede transmitir. Por ejemplo, en Java, un

objeto de excepción puede proporcionar toda esta información de manera encapsulada, mientras que en C sería necesario implementar manualmente mecanismos adicionales para lograr algo similar.

8. En Java, sobre el bloque "try-catch", ¿se pueden tener más de un bloque `catch`? ¿cuántos bloques `catch` se ejecutan?

Respuesta

En Java, es posible tener múltiples bloques `catch` asociados a un único bloque `try`. Cada bloque `catch` maneja un tipo específico de excepción, lo que permite tratar diferentes errores de manera diferenciada.

Sin embargo, solo se ejecuta un bloque `catch`, el primero que coincide con el tipo de la excepción lanzada. Una vez que se ejecuta un bloque `catch`, los demás se omiten. Por ejemplo:

```
try {
    // Código que puede lanzar excepciones
} catch (IOException e) {
    System.out.println("Error de entrada/salida: " + e.getMessage());
} catch (ArithmeticException e) {
    System.out.println("Error aritmético: " + e.getMessage());
}
```

En este caso, si se lanza una excepción de tipo `IOException`, solo se ejecutará el primer bloque `catch`, incluso si hay otros bloques `catch` definidos.

9. Si las excepciones producen rupturas en el código llamador, ¿cómo podemos garantizar que se ejecuta siempre finalmente un código necesario para cierre de ficheros, liberacion de recursos, antes de que continúe propagándose la excepción? Pon un

ejemplo en Java con `finally`, tanto con `catch` como sin él.

Respuesta

En Java, el bloque `finally` se utiliza para garantizar que un conjunto de instrucciones se ejecute siempre, independientemente de si ocurre o no una excepción. Esto es útil para liberar recursos, cerrar archivos o realizar tareas de limpieza.

Ejemplo con `catch`:

```java
try {
    // Código que puede lanzar excepciones
    int resultado = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("Liberando recursos...");
}
```

Ejemplo sin `catch`:

```java
try {
    int resultado = 10 / 0;
} finally {
    System.out.println("Este bloque siempre se ejecuta.");
}
```

En ambos casos, el bloque `finally` se ejecuta siempre, incluso si ocurre una excepción o si no hay un bloque `catch`.

10. En Java, el bloque `finally` puede ir sin `catch`? ¿Se ejecuta siempre tanto si ocurre como si no ocurre

una excepción? ¿Y si hay un `return` en medio del `try`?

Respuesta

Sí, el bloque `finally` puede ir sin un bloque `catch`. Este bloque se ejecuta siempre, independientemente de si ocurre o no una excepción. Incluso si hay un `return` dentro del bloque `try`, el bloque `finally` se ejecutará antes de que el método termine.

Por ejemplo:

```
public static void ejemplo() {
    try {
        System.out.println("Dentro del try");
        return;
    } finally {
        System.out.println("Dentro del finally");
    }
}

public static void main(String[] args) {
    ejemplo();
}
```

La salida será:

```
Dentro del try
Dentro del finally
```

Esto demuestra que el bloque `finally` se ejecuta incluso cuando hay un `return` en el bloque `try`.

11. En Java, qué son las excepciones "controladas" y las "no controladas"? ¿Qué papel juega `RuntimeException`? Pon un ejemplo de excepciones típicas controladas y no controladas que incluso nosotros mismos podríamos usar. Haz dos listas con

3 o 4 ejemplos de situación donde se suele preferir una excepción controlada y donde se suele preferir una excepción no controlada.

Respuesta

Las excepciones controladas (checked exceptions) son aquellas que el compilador obliga a manejar explícitamente, ya sea capturándolas con un bloque `try-catch` o declarándolas con `throws`. Ejemplo: `IOException`.

Las excepciones no controladas (unchecked exceptions) son subclases de `RuntimeException` y no requieren manejo explícito. Ejemplo: `NullPointerException`.

Ejemplos de excepciones controladas:

- `IOException`
- `SQLException`
- `FileNotFoundException`

Ejemplos de excepciones no controladas:

- `NullPointerException`
- `ArithmeticException`
- `IllegalArgumentException`

Se prefieren excepciones controladas para errores que el programador puede anticipar y manejar, como problemas de entrada/salida. Las excepciones no controladas se usan para errores de programación, como referencias nulas.

12. ¿Qué es y para qué se usa `throws`? ¿Por qué es alternativa a capturar una excepción controlada?

Respuesta

La cláusula `throws` se utiliza en la firma de un método para declarar que este puede lanzar ciertas excepciones. Esto permite propagar la excepción al método llamador en lugar de capturarla dentro del método.

Por ejemplo:

```
public void leerArchivo(String nombreArchivo) throws IOException {
    FileReader lector = new FileReader(nombreArchivo);
}
```

En este caso, el método `leerArchivo` declara que puede lanzar una `IOException`, dejando la responsabilidad de manejarla al método que lo llame.

13. Pon un ejemplo en Java de firma de método que incluya `throws`, de una función que abre un fichero pero que declara que no le interesa manejar la excepción de si el fichero no existe, sino que se propague hacia arriba. Eso sí, acuérdate del `finally`.

Respuesta

```
public void abrirArchivo(String nombreArchivo) throws FileNotFoundException {
    FileReader lector = null;
    try {
        lector = new FileReader(nombreArchivo);
    } finally {
        System.out.println("Intento de abrir archivo finalizado.");
    }
}
```

En este ejemplo, la excepción `FileNotFoundException` se propaga al método llamador, pero el bloque `finally` asegura que se ejecuten las tareas de limpieza.

14. ¿Podemos poner en `throws` excepciones no controladas, como `RuntimeException`? ¿Debería el

método llamador entonces poner `try-catch` en ese caso? ¿Qué sentido tendría?

Respuesta

Sí, es posible declarar excepciones no controladas en `throws`, aunque no es obligatorio. Esto puede ser útil para documentar que un método puede lanzar una excepción específica.

El método llamador no está obligado a manejarla, pero puede hacerlo si desea evitar que el programa termine abruptamente. Por ejemplo:

```
public void dividir(int a, int b) throws ArithmeticException {
    if (b == 0) {
        throw new ArithmeticException("División por cero");
    }
    System.out.println(a / b);
}
```

15. ¿Cuándo se recomienda usar excepciones controladas, como `IOException`, y cuándo no controladas como `IllegalArgumentException`? ¿Existen en todos los lenguajes ambas opciones? En los que sólo existe una opción, ¿cuál es la más habitual?

Respuesta

Las excepciones controladas se recomiendan para errores que el programador puede anticipar y manejar, como problemas de entrada/salida. Las no controladas se usan para errores de programación, como argumentos inválidos.

No todos los lenguajes distinguen entre excepciones controladas y no controladas. En lenguajes como Python, todas las excepciones son similares a las no controladas de Java.

16. ¿Tiene sentido lanzar excepciones dentro del `catch`? ¿Se puede relanzar la misma excepción capturada? ¿Cuándo tendría sentido hacer esto último? Pon ejemplos de ambos casos.

Respuesta

Lanzar excepciones dentro de un `catch` tiene sentido cuando se desea encapsular un error en una excepción más específica o agregar contexto adicional. Relanzar la misma excepción es útil cuando no se puede manejar completamente el error en el nivel actual.

Ejemplo de lanzar una nueva excepción:

```
try {
    // Código que lanza IOException
} catch (IOException e) {
    throw new RuntimeException("Error al procesar archivo", e);
}
```

Ejemplo de relanzar la misma excepción:

```
try {
    // Código que lanza IOException
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
    throw e;
}
```

17. ¿En qué consiste que una excepción sea la "causa" de otra excepción? Pon un ejemplo en Java, donde capturemos una excepción de bajo nivel y la encapsulemos en otra personalizada de alto nivel.

Cuando una excepción sale por pantalla y tiene una causa, ¿se ve?

Respuesta

Una excepción puede ser la causa de otra cuando se encapsula dentro de una nueva excepción. Esto permite preservar el contexto del error original mientras se agrega información adicional.

Ejemplo:

```
try {
    // Código que lanza FileNotFoundException
} catch (FileNotFoundException e) {
    throw new MiExcepcion("Archivo no encontrado", e);
}
```

Cuando se imprime la excepción, la causa aparece en la traza de la pila, proporcionando información completa sobre el error original y el contexto adicional.